Sravya Madipalli

Master Data Cleaning in SQL

Data cleaning is a critical step in any data analysis or data science project. Without proper data cleaning, your analysis may lead to inaccurate or misleading results.

Today we will look into -

- Essential SQL data cleaning techniques
- Practical examples to demonstrate each concept
- Step-by-step strategies to help you clean and prepare your data effectively

At the end, you'll also find common interview questions to test your knowledge and readiness for SQL-focused roles.

Handling Missing Values

Missing values can lead to inaccurate analysis or cause errors during joins and aggregations. SQL provides several ways to deal with missing or null values.

Solution

Use COALESCE() or IFNULL() to replace missing values with defaults.

Code Example:

```sql
SELECT COALESCE(email, 'unknown') AS cleaned_email
FROM users;
```

Explanation:

- COALESCE() returns the first non-null value from a list of arguments.
- This query replaces any NULL values in the email column with 'unknown', ensuring data integrity.

Removing Duplicates

Duplicates in data can distort results and lead to incorrect conclusions. SQL offers multiple ways to identify and remove duplicates.

Solution: Use DISTINCT or ROW_NUMBER() to eliminate duplicate rows.

Code Example:

```sql
WITH RankedRows AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
created_at DESC) AS row_num
  FROM orders
)
SELECT * FROM RankedRows
WHERE row_num = 1;
```

Explanation:

- ROW_NUMBER() assigns a unique number to each row within a partition defined by user_id.
- This query keeps only the most recent row for each user_id, removing older duplicates.
- Useful when tracking unique users or transactions.

Standardizing Data Formats

Inconsistent data formats, especially with text, can cause issues when performing comparisons or analysis.

Solution: Use UPPER(), LOWER(), and TRIM() to standardize text formats.

Code Example:

```sql
SELECT LOWER(first_name) AS standardized_name
FROM customers;
```

Explanation:

- LOWER() converts text to lowercase, ensuring consistent formatting.
- This is especially useful when performing case-sensitive comparisons, avoiding mismatches due to inconsistent capitalization.

Handling Outliers

Outliers can distort the results of your analysis, affecting averages, totals, and other metrics. Proper handling of outliers is crucial.

Step 1: Identifying Outliers:

Use statistical measures like AVG() and STDDEV() to detect outliers

Code Example:

```sql
SELECT order_id, amount
FROM orders
WHERE amount > (SELECT AVG(amount) + 3 *
STDDEV(amount) FROM orders);
```

Explanation:

- This query identifies any amount values that are more than three standard deviations above the average.
- Such extreme values often represent outliers that can skew your analysis.

Handling Outliers

Step 2: Removing or Capping Outliers

Use statistical measures like AVG() and STDDEV() to detect outliers

Removing Outliers:

```sql
DELETE FROM orders
WHERE amount > (SELECT AVG(amount) + 3 *
STDDEV(amount) FROM orders);
```

Capping Outliers:

```sql
UPDATE orders
SET amount = (SELECT AVG(amount) + 3 * STDDEV(amount) FROM orders)
WHERE amount > (SELECT AVG(amount) + 3 * STDDEV(amount) FROM orders);
```

Explanation:

- Removing: Deletes rows where amount exceeds the outlier threshold.
- Capping: Limits the value of amount to a maximum value, reducing the effect of outliers while preserving the row.

Dates-Related Data Cleaning

Dates are critical for time-based analysis. Standardizing date formats and extracting specific components are common tasks.

Standardizing Date Formats

Ensure all dates follow a consistent format using functions like TO_DATE().

```sql
SELECT TO_DATE(order_date, 'YYYY-MM-DD') AS standardized_date
FROM orders;
```

Explanation:

- TO_DATE() converts a variety of date formats into a standard YYYY-MM-DD format, ensuring consistency.

Dates-Related Data Cleaning

Extracting Year, Month, or Day from Dates

Sometimes you need to break down a date into its components for specific analyses, like grouping by year or month.

```sql
SELECT EXTRACT(YEAR FROM order_date) AS year,
       EXTRACT(MONTH FROM order_date) AS month
FROM orders;
```

Explanation:

- EXTRACT() pulls out individual components like year or month from a date field.
- This is useful for time-based aggregations and identifying trends over specific periods.

Correcting Data Entry Errors

Manual data entry often leads to errors in format, especially in fields like phone numbers or emails. These errors can cause issues downstream in your analysis.

Solution: Use REGEXP to detect and correct formatting errors.

Code Example:

Explanation:

- REGEXP is a regular expression function that allows you to match patterns.
- This query finds phone numbers that don't match the 10-digit numeric format.
- By detecting such inconsistencies early, you can avoid analysis errors later on.

Handling Null Values in Aggregations

Null values in aggregations can cause incorrect results, as they may be excluded from counts, sums, or averages.

Solution: Use COALESCE() or modify aggregation functions to handle nulls.

Code Example:

```sql
SELECT SUM(COALESCE(amount, 0)) AS total_amount
FROM orders;
```

Explanation:

- COALESCE() replaces NULL values with 0 before summing the amount.
- This ensures that null values do not lead to inaccurate totals

Removing Leading and Trailing Spaces

Extra spaces can cause comparison issues and lead to inconsistent results, especially in text fields.

Solution: Use TRIM() to remove unnecessary whitespace.

Code Example:

```sql
SELECT TRIM(first_name) AS trimmed_name
FROM employees;
```

Explanation:

- TRIM() removes leading and trailing spaces, ensuring consistent and clean data for comparisons and joins.

Data Cleaning Interview Questions

1. How would you handle missing values in a dataset?
    a. Discuss techniques such as using COALESCE() or replacing nulls with averages or other default values.
2. What is the difference between removing and capping outliers, and when would you use each?
    a. Explain how removing outliers completely eliminates them, while capping reduces their impact without removing the data point.
3. Can you explain how you would standardize date formats in SQL?
    a. Walk through the process of using TO_DATE() or similar functions to ensure consistency in date formats.
4. How can you remove duplicates from a dataset?
    a. Discuss using DISTINCT or ROW_NUMBER() to identify and eliminate duplicates.
5. What methods can you use to identify and correct data entry errors, like incorrectly formatted phone numbers?
    a. Explain how REGEXP can be used to identify patterns and detect inconsistencies.
6. Why is it important to handle null values in aggregations, and how would you do it in SQL?
    a. Mention COALESCE() or handling nulls directly in aggregate functions like SUM() or COUNT().

Sravya Madipalli

Was this Helpful?

- Save it
- Follow Me
- Repost and Share it with your friends